

**INSTITUTO
FEDERAL**

Goiano

Campus
Trindade

Programação Web

Tokenização com JWT

Node.js · Express · PostgreSQL · JWT

Prof. Dr. Renato de Sousa Gomide

Objetivos da aula

- Entender **por que** APIs precisam provar quem é o usuário a cada requisição
- Conhecer **JWT** (*JSON Web Token*) e suas **três partes**
- Implementar **registro** com senha hashada (**bcrypt**) e **login** com emissão de token
- Usar **middleware** no Express para **proteger rotas**
- Testar a API com **Authorization: Bearer** e aplicar **boas práticas de segurança**

O problema: quem é você?

Imagine um prédio com **catraca** em cada andar. Sem crachá, você teria que mostrar documento **toda hora**.

Em APIs web, o equivalente ao crachá é provar **identidade** a cada requisição — sem o servidor guardar sessão de cada usuário conectado.

Neste módulo: rotas de **alunos** que antes eram públicas passam a exigir **login**.

Sessão no servidor vs JWT

Abordagem	Como funciona	Prós	Contras
Sessão	Servidor guarda quem está logado (memória, Redis, banco)	Revogação imediata	Mais estado; escalar exige compartilhar sessões
JWT (stateless)	Servidor emite token assinado ; cliente envia a cada requisição	Simples de escalar; ideal para APIs e SPAs	Revogar antes do vencimento exige estratégia extra

JWT = autenticação **stateless**: o servidor **não guarda** lista de tokens válidos.

O que é um JWT?

JWT (*JSON Web Token*) é um padrão ([RFC 7519](#)) para representar informações de forma **compacta** e **assinada**.

- Analogia:
 - é como um **ingresso de show com holograma**. O organizador (servidor) emite
 - a portaria (middleware) confere se o holograma bate com o que só o organizador sabe fabricar

Se alguém alterar o conteúdo, a **assinatura** deixa de bater — a falsificação é detectada.

Stack do projeto



Node.js + Express



PostgreSQL



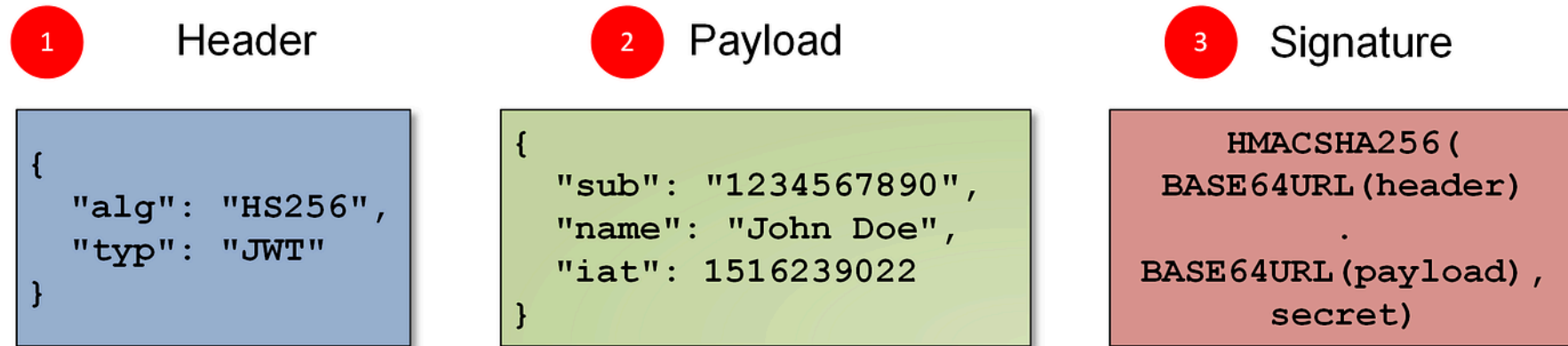
jsonwebtoken

Bibliotecas principais:

- **jsonwebtoken** (criar/validar tokens)
- **bcryptjs** (hash de senhas)

Anatomia de um token

1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzkwMjQ.DIyfQ.XbPfbIHMI6arZ3Y922BhjWgQzWXcXNrZ0ogtVhfEd2o 2 3



Três partes em **Base64URL**, separadas por **ponto** (.).

As três partes em detalhe

Parte	Conteúdo típico
Header	Algoritmo (ex.: <code>HS256</code>) e tipo <code>JWT</code>
Payload	<i>Claims</i> : <code>id</code> , <code>email</code> , <code>iat</code> (emitido em), <code>exp</code> (expira em)
Signature	Assinatura de header + payload com <code>JWT_SECRET</code>

- O **payload não é criptografado** — apenas **codificado**
- Qualquer um pode **ler**
- A assinatura impede **alterar** sem ser detectado.

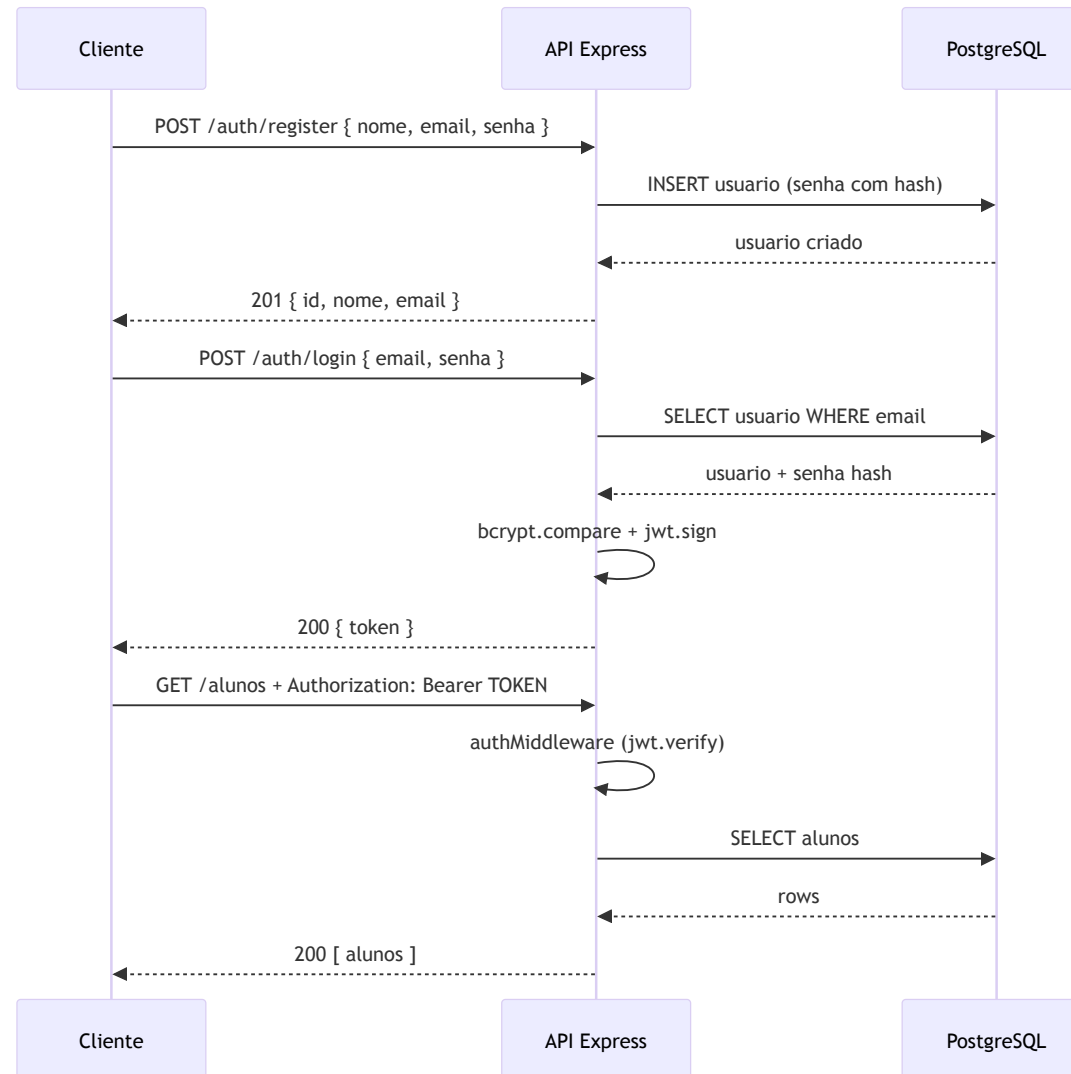
O que **não** colocar no payload

- **Senha** (nem hasheada)
- Dados sensíveis: cartão, CPF completo, etc.

Coloque só o **mínimo** para identificar o usuário: `id`, `email`, `nome`.

Dúvida comum: "JWT é seguro?" — Sim para **integridade** e **autenticidade**, desde que o segredo permaneça no servidor. Mas o conteúdo é **legível** — trate como um crachá visível, não como cofre.

Fluxo de autenticação



O que é middleware?

Até aqui, cada rota era `(req, res) => { ... }`. **Middleware** intercepta a requisição **antes** do handler final.

```
function exemploMiddleware(req, res, next) {  
  // req  - requisição (headers, body...)  
  // res  - resposta  
  // next - passa adiante (ou a requisição fica "presa")  
  next();  
}
```

Analogia: **triagem** na fila antes do guichê. Algo errado? Manda embora (`401`). OK?
`next()` .

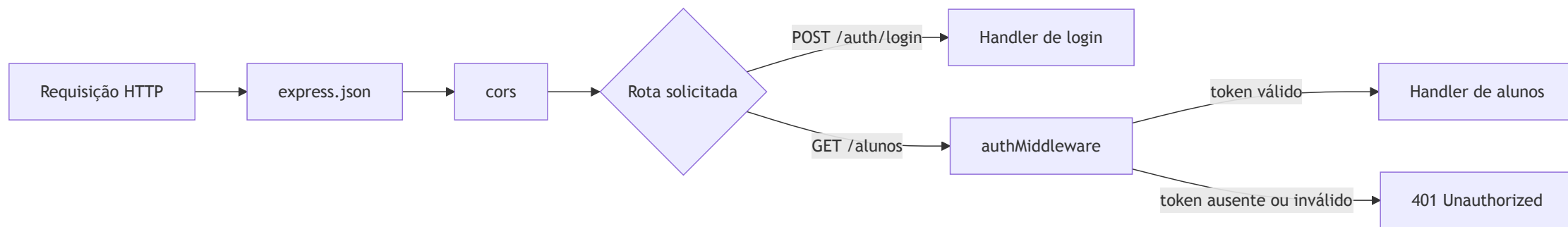
Middleware global vs por rota

Forma	Sintaxe	Quando roda
Global	<code>app.use(middleware)</code>	Em toda requisição
Por rota	<code>app.get('/caminho', middleware, handler)</code>	Só naquela rota, antes do handler

Você já usou middleware global no módulo Express:

```
app.use(express.json()); // body JSON → req.body
app.use(cors());         // cabeçalhos CORS
```

Pipeline deste projeto



authMiddleware só entra em **GET /alunos** , **GET /alunos/:id** e **GET /auth/me** .

Rotas de **login** e **registro** permanecem **públicas**.

Registro — senha com hash

A senha **nunca** vai ao banco em texto puro. Guardamos um **hash** — transformação **unidirecional**.

```
const senhaHash = await bcrypt.hash(senha, SALT_ROUNDS);

const result = await query(
  'INSERT INTO usuario (nome, email, senha) VALUES ($1, $2, $3) RETURNING id, nome, email, created_at',
  [nome, email, senhaHash]
);
```

RETURNING devolve o usuário **sem** o campo **senha** na resposta da API.

O que é *salt*?

- *Salt* é um valor **aleatório** gerado para **cada senha** antes do hash. O bcrypt gera o salt automaticamente e o **embute** no hash final.
- Analogia: salgar um prato antes de cozinhar. Dois pratos iguais (mesma senha `123456`) ficam com "sabor" diferente — um atacante não descobre senhas iguais comparando hashes.
- No login: `bcrypt.compare(senha, usuario.senha)` repete o processo com o salt salvo.

Login — emissão do token

Após validar email e senha, o servidor **assina** um payload mínimo:

```
const token = jwt.sign(  
  { id: usuario.id, nome: usuario.nome, email: usuario.email },  
  process.env.JWT_SECRET,  
  { expiresIn: process.env.JWT_EXPIRES_IN || '1h' }  
);  
  
return res.json({ token });
```

O cliente **guarda** o token e o envia nas próximas requisições.

Cabeçalho Authorization: Bearer

Padrão mais usado em APIs REST:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

- **Bearer** = "quem porta este token"
- O token vem **depois** do espaço
- Sem esse cabeçalho → **401 Token não informado**

Middleware de autenticação

Arquivo `src/middleware/auth.js` — a **portaria** do JWT:

```
const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ erro: 'Token não informado' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.usuario = payload; // disponível no handler
    next();
  } catch (error) {
    return res.status(401).json({ erro: 'Token inválido ou expirado' });
  }
};
```

Três detalhes do middleware

1. `return res.status(...)` — encerra aqui; **não** chama `next()` → handler nunca roda
2. `req.usuario = payload` — middleware **enriquece** `req` para as rotas seguintes
3. `next()` — só quando token é **válido** e **não expirado**

Token ausente, malformado, assinado com segredo errado ou **expirado** → **401 Unauthorized**.

Protegendo rotas

Basta **inserir o middleware** entre a rota e o handler:

```
app.get('/alunos', authMiddleware, async (req, res) => {  
  const alunos = await alunoModel.getAll();  
  return res.json(alunos);  
});
```

Sintaxe: `app.METODO(caminho, middleware, handler)`. Vários middlewares? Express executa **da esquerda para a direita**.

Rotas da API

Método	Caminho	Auth	Descrição
GET	/	Não	Informações sobre a API
POST	/auth/register	Não	Cadastra usuário
POST	/auth/login	Não	Retorna JWT
GET	/auth/me	Sim	Dados do usuário logado
GET	/alunos	Sim	Lista alunos
GET	/alunos/:id	Sim	Busca aluno por ID

Configuração (`.env`)

Variável	Descrição
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_NAME</code>	Conexão PostgreSQL
<code>JWT_SECRET</code>	Segredo longo e aleatório para assinar tokens
<code>JWT_EXPIRES_IN</code>	Validade (ex.: <code>1h</code> , <code>15m</code>)

Gerar segredo seguro:

```
npm run generate:jwt-secret
```

Nunca commite o `.env` real. Use `npm run dev` para subir na porta **3000**.

Boas práticas de segurança

1. **JWT_SECRET forte** — `npm run generate:jwt-secret`
2. **HTTPS em produção** — token trafega a cada requisição
3. **Expiração curta** — `JWT_EXPIRES_IN` de horas ou minutos
4. **Payload enxuto** — busque dados frescos no banco (`GET /auth/me`)
5. **Senhas com hash** — bcrypt; nunca logue senhas ou tokens completos
6. **Revogação** — JWT sozinho não "desloga" no servidor até expirar

Autenticação vs Autorização

O `authMiddleware` resolve só metade do problema: ele prova **quem** está logado (**autenticação**). Decidir **o que** essa pessoa pode fazer é **autorização** — uma camada separada, construída por cima.

Exemplo: João e Maria autenticados com tokens válidos. João tenta editar o cadastro da Maria — token válido, mas ação **não autorizada**.

Situação	Status
Token ausente/inválido	401 (autenticação)
Token válido, ação não permitida	403 (autorização)

Resumo

- **JWT** = crachá assinado pelo servidor; cliente envia a cada requisição
- **Header · Payload · Signature** — payload legível, assinatura garante integridade
- **bcrypt** hasheia senhas no registro; **bcrypt.compare** no login
- **authMiddleware** valida token e preenche `req.usuario`
- Rotas protegidas: `app.get('/rota', authMiddleware, handler)`

Próximos passos

- Distinguir token **expirado** vs **inválido** (código `TOKEN_EXPIRADO`)
- Tópicos avançados: *refresh token*, blacklist, logout imediato

Exercícios propostos

1. **Proteger criação de aluno** — `POST /alunos` só para autenticados
2. **Autorização** — usuário só atualiza o **próprio** aluno (`PUT /alunos/:id` + `id_aluno`)
3. **Middleware opcional** — `authOptional` preenche `req.usuario` se houver token, sem bloquear
4. **Validação de senha** — mínimo 8 caracteres + 1 número no registro
5. **Refresh de perfil** — `PUT /auth/me` emite **novo** JWT após atualizar nome
6. **Token expirado vs inválido** — distinguir `TokenExpiredError` com código `TOKEN_EXPIRADO`

Referências

- jwt.io — decodificar e entender tokens
- [RFC 7519 — JSON Web Token](#)
- Bibliotecas principais:
 - [jsonwebtoken](#)
 - [bcryptjs](#)